RICE UNIVERSITY

Arithmetic Intensity and the Roofline Model

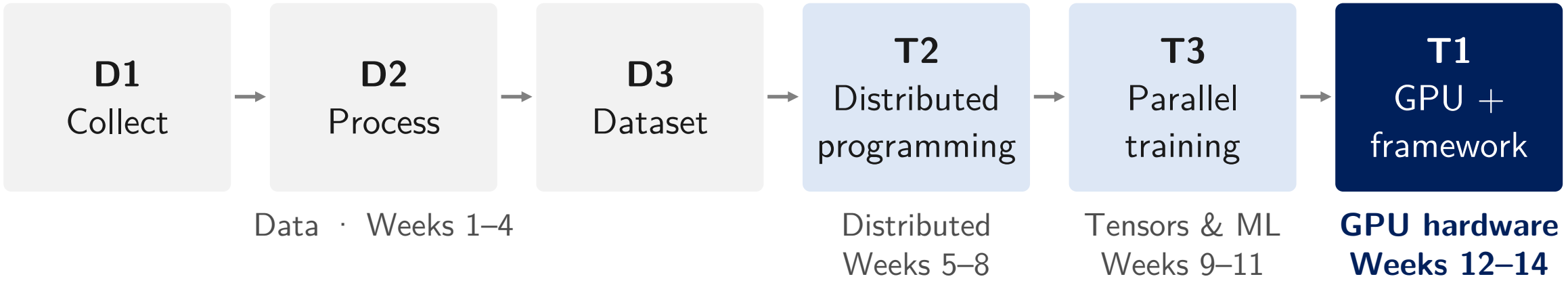
Count the FLOPs, count the bytes, and predict whether arithmetic or memory limits a kernel

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 13, Lecture 2

Where we are in the pipeline



In our pipeline

Today: one picture that says whether **arithmetic** or **memory** limits a kernel — the prediction **T1** asks for. T1 checkpoint this week.

Lecture 1: where bytes come from → today: bytes versus FLOPs → Week 14: a fast matrix multiply

Why do we need a performance model?

A slow kernel: $y = a \cdot x + b$, $n = 10^8$ floats

Proposal: make its **arithmetic 10× faster**

How much faster does the kernel get?

Arithmetic: 2×10^8 FLOPs

$\div$ 67 TFLOP/s $\approx$ **3 µs**

Memory: 8×10^8 bytes

$\div$ 3.35 TB/s $\approx$ **0.24 ms**

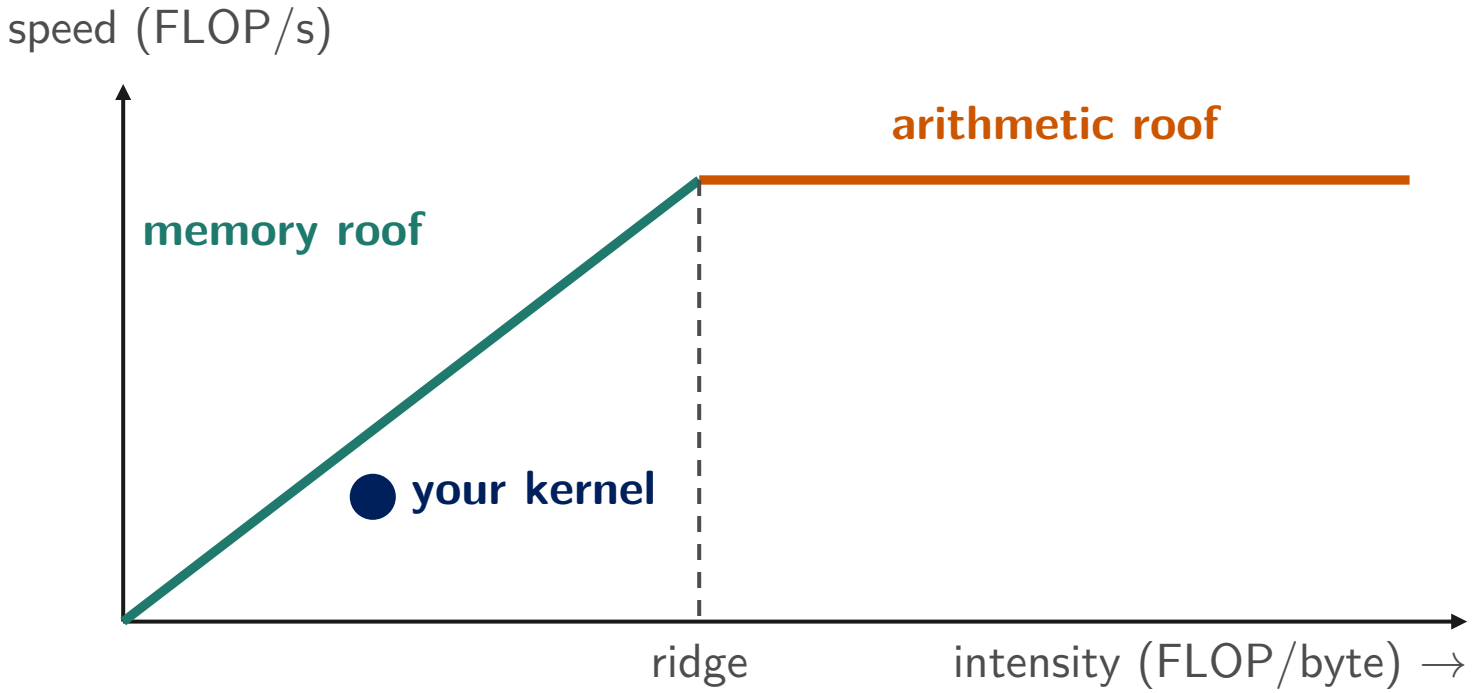
Answer

Barely at all: about **1%**. The kernel waits on **memory**. Know which limit you are against **before** you optimize.

Source: H100 datasheet (SXM5): FP32 peak and HBM bandwidth.

The roofline model, in plain words

Arithmetic intensity: FLOPs done per byte moved from memory. **Roofline**: the best speed a GPU can reach, as a function of that intensity.



Left of the ridge: memory limits you.

Right of the ridge: arithmetic limits you.

Where the roofline shows up

- **2009, Berkeley**: Williams, Waterman & Patterson, for multicore CPUs.
- **Nsight Compute**, NVIDIA's profiler, draws one for every kernel.
- **FlashAttention**: attention is memory-bound $\rightarrow$ move fewer bytes.
- **T1**: predict your kernel's traffic and intensity, then measure.

The price

A **bound**, not an exact time

Only as good as your **counts** of F and Q

Ignores **launch** overhead

In our pipeline

Why a roofline? Two numbers about the GPU and two about the kernel tell you **which fix can help** — before you write or run anything.

Source: NVIDIA Nsight Compute documentation (roofline charts); Williams, Waterman & Patterson, CACM 2009.

Learning goals for today

- Count **FLOPs F** and minimum **bytes Q**; compute the **intensity I = F / Q**.
- Derive the roofline $\mathbf{T \geq \max(F/\pi, Q/\beta)}$ and find the **ridge point**.
- Classify operations as **memory-bound** or **compute-bound**, and say which fix helps.
- Predict a kernel's bottleneck and runtime **before** measuring it.

A puzzle to start

Kernel A — vector add

$z = x + y$, $n = 10^8$ floats

1 add per element

Kernel B — matrix multiply

$C = A \cdot B$, 4096×4096

8,192 FLOPs per output

On the same GPU, which kernel reaches a larger fraction of the advertised peak FLOP/s?

Key idea

Kernel B gets close to peak; **kernel A** reaches well under 1% of it. For A, arithmetic is not the bottleneck — **moving the data is**.

Part 1

Counting work and bytes

How much arithmetic, and how much data, does a kernel need?

Counting work: FLOPs

- **FLOP** = one floating-point add or multiply; **FLOP/s** is a rate.
- Dot product of length K: K multiplies + (K − 1) adds ≈ **2K FLOPs**.
- C = A · B with A (M × K), B (K × N): M · N dot products.

$$F = 2 \cdot M \cdot N \cdot K$$

$$M = N = K = 4096$$

$$F \approx 1.37 \times 10^{11} \text{ FLOPs}$$

Compute

At 989 TFLOP/s this takes ≈ **0.14 ms** — *if* arithmetic were the only cost.

Source: H100 datasheet (SXM5), BF16 tensor-core peak without sparsity.

What counts as a FLOP?

Operation	FLOPs
add, multiply	1
fused multiply-add $a \cdot b + c$	2
compare / max (ReLU)	1
exp, tanh (course convention)	1

Key idea

For a **memory-bound** kernel the predicted time is Q/β: the FLOP convention does not change it.

Counting bytes: the minimum traffic Q

- **Q** = bytes between HBM and the chip.
- Minimum: every input **read once**, every output **written once**.
- FP32 = **4** bytes, BF16 = **2** bytes per number.

$$\text{vector add:} \quad Q = 3 \times 4 \times n = 12n$$

$$n = 10^8\text{:} \quad Q = 1.2 \times 10^9 \text{ bytes}$$

$$n \times n \text{ matmul:} \quad Q = 3n^2 \text{ numbers}$$

Memory

Q is a **lower bound**: uncoalesced access (Week 13, L1) or missing reuse only add to it.

Arithmetic intensity

$$I = F \,/\, Q$$

$$F = \text{FLOPs performed}, \quad Q = \text{bytes moved to/from HBM}$$

Kernel	F	Q (minimum)	I (FLOP/B)
Vector add z = x + y (FP32)	n	12n	$1/12 \approx 0.08$
Row softmax, B × V (FP32)	$\approx 5BV$	$8BV$	≈ 0.6
Matmul n × n (BF16, ideal reuse)	$2n^3$	$6n^2$	n / 3

Key idea

Elementwise and reduction kernels: **I below 1**, whatever the size. Matmul: I **grows** with n.

Why matmul's intensity grows

$$F = 2n^3 \qquad \text{(work grows like } n^3\text{)}$$

$$Q = 3 \cdot n^2 \cdot 2 \text{ bytes} = 6n^2 \qquad \text{(data grows like } n^2\text{)}$$

$$I = F / Q = n / 3$$

n	**256**	**1024**	**4096**
I (FLOP/B)	85	341	1365

Key idea

Each input is used **n times**. "Ideal reuse" assumes it is read from HBM **once** — how to get there is Week 14, L1.

Counting work for our own model

Step (batch B = 64)	FLOPs
Hidden: (64×128) · (128×64), tanh	≈ 1.05 M
Logits: (64×64) · (64×322)	≈ 2.64 M
Backward ≈ 2 × matmuls	≈ 7.4 M
One training step	**≈ 11 MFLOP**

ModelConfig: vocab 322, context 8, embedding 16, hidden 64

34,338 parameters ≈ 137 KB (FP32)

Check

At 67 TFLOP/s, 11 MFLOP takes ≈ **0.17 µs**. Hold that thought.

Check yourself

Two minutes with a neighbour.

- Scale a vector, $y = 2x$, n floats in FP32: F, Q, and I?
- The same kernel in BF16: what changes, and why?
- Matmul with $n = 1024$ in BF16, ideal reuse: I?

Part 2

The roofline model

Compute-bound or memory-bound? Predict before you measure.

Two lower bounds on runtime

$$T \geq F \,/\, \pi \qquad \text{(arithmetic)}$$

$$T \geq Q \,/\, \beta \qquad \text{(memory)}$$

$$T \geq \max(\, F \,/\, \pi \;,\;\; Q \,/\, \beta \,)$$

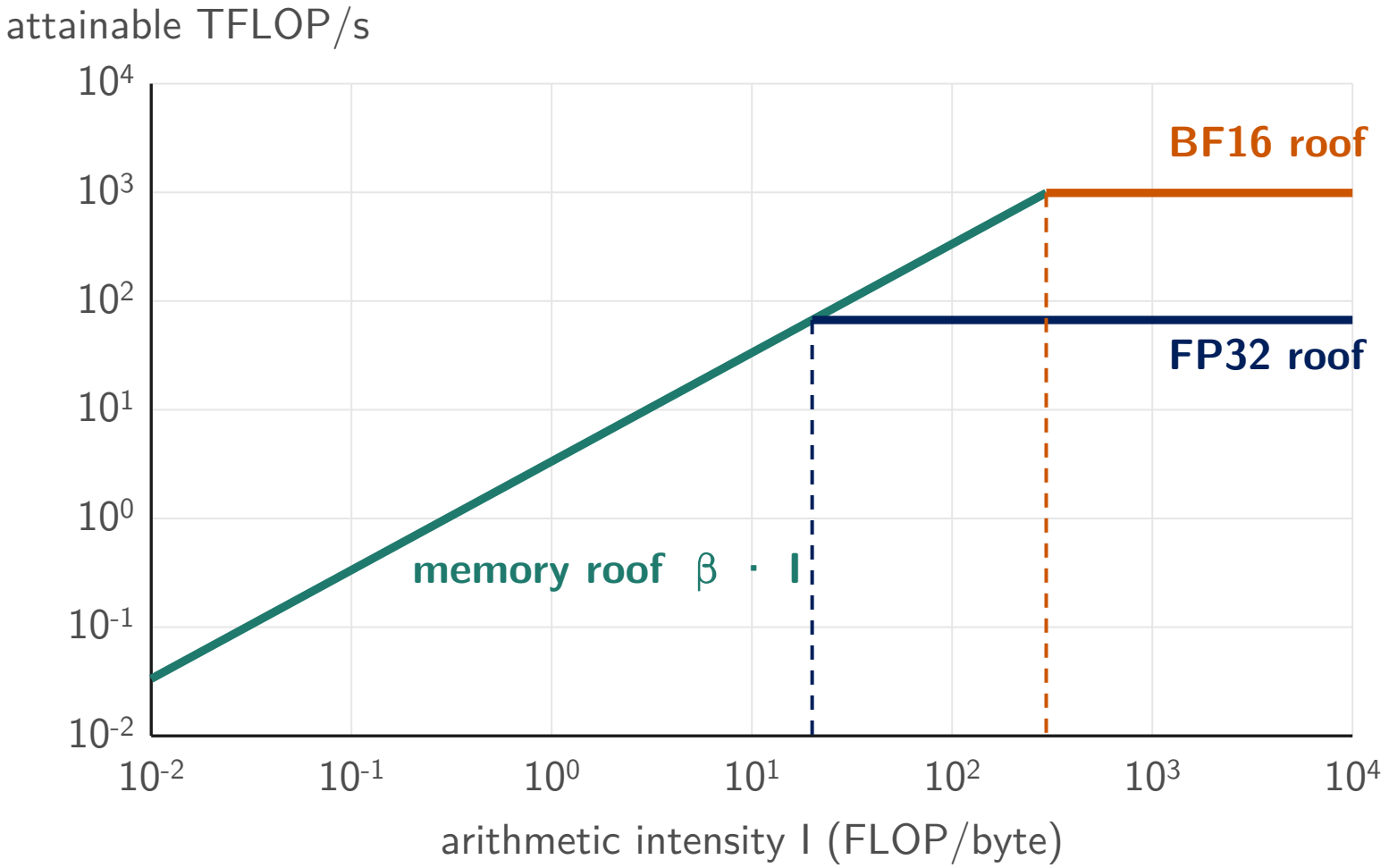
$$\text{attainable FLOP/s} = F \,/\, \; T \leq \min(\, \pi \;,\;\; \beta \cdot I \,)$$

Key idea

Ridge point $I^* = \pi \,/\, \beta$: below it, memory limits you; above it, arithmetic does. Assumes perfect overlap — an optimistic bound.

Source: Williams, Waterman & Patterson, "Roofline," Communications of the ACM 52(4), 2009; H100 datasheet (SXM5).

The roofline for an H100



FP32 roof: 67 TFLOP/s

BF16 roof: 989 TFLOP/s

ridges $\approx$ 20 and 295

$\leftarrow$ memory-bound: move fewer bytes

compute-bound $\rightarrow$ use faster arithmetic

Source: Williams, Waterman & Patterson, "Roofline," Communications of the ACM 52(4), 2009; H100 datasheet (SXM5).

Which fix can help?

Memory-bound: left of the ridge

Move fewer bytes:

- coalesce (Week 13, L1)
- reuse on chip (Week 14, L1)
- fuse kernels (Week 14, L2)

Faster arithmetic: **no help**

Compute-bound: right of the ridge

Use faster arithmetic:

- tensor cores
- lower precision (Week 14, L1)

Fewer bytes: **no help**

Watch out

Optimizing the **wrong** limit buys nothing. Find the limit first.

Worked example 1: vector add

- $n = 10^8$ floats (FP32); $F = 10^8$ FLOPs
- $Q = 3 \times 4 \times 10^8 = 1.2 \times 10^9$ bytes
- $I = F / Q = 1/12 \approx 0.08$ FLOP/B $(\ll 20)$
- $F / \pi = 10^8 / 67 \times 10^{12} \approx 1.5$ µs
- $Q / \beta = 1.2 \times 10^9 / 3.35 \times 10^{12} \approx$ **0.36 ms**

Verdict

Memory-bound by $\approx 240\times$. A measured time near 0.36 ms means the kernel is already **optimal**: it runs at the memory roof.

Worked example 2: matrix multiply

$$F = 2n^3, \quad Q = 6n^2, \quad I = n \,/\, 3$$

$$\text{compute-bound when } n \,/\, 3 > 295 \;\Leftrightarrow\; n \gtrsim 886$$

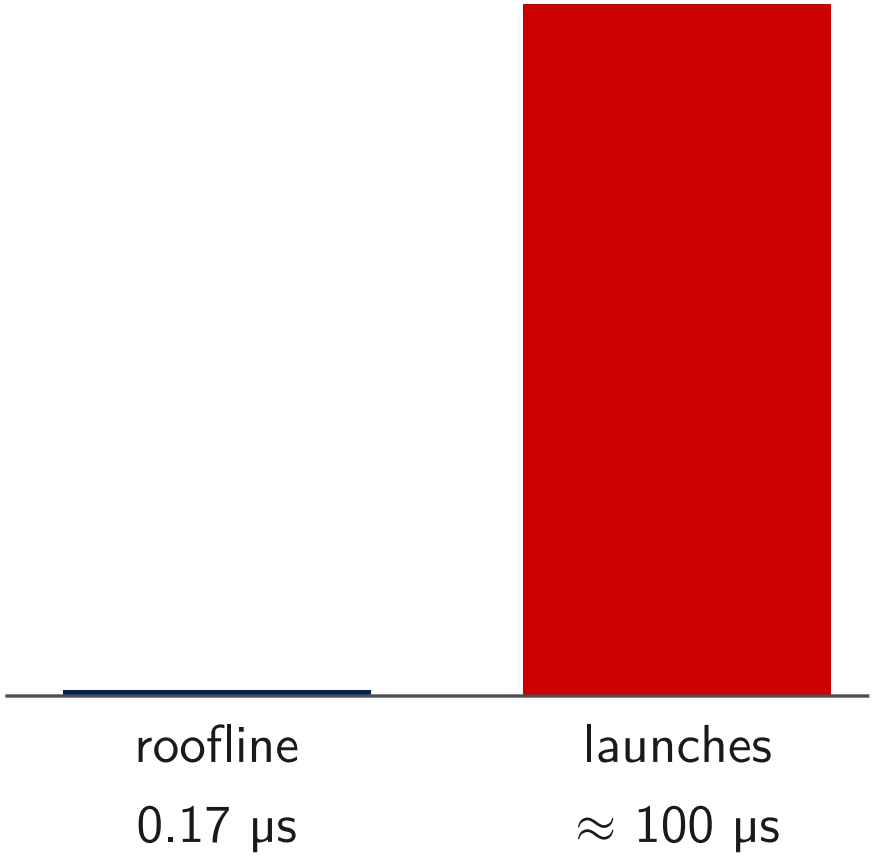
n	I (FLOP/B)	Bound	Predicted time
256	85	memory (on paper)	0.12 µs → launch overhead dominates
1024	341	compute (just)	≈ 2.2 µs
4096	1365	compute	≈ 0.14 ms

Watch out

Fixed overhead is a third limit: a launch costs several µs (Week 12, L2).

Worked example 3: our training step

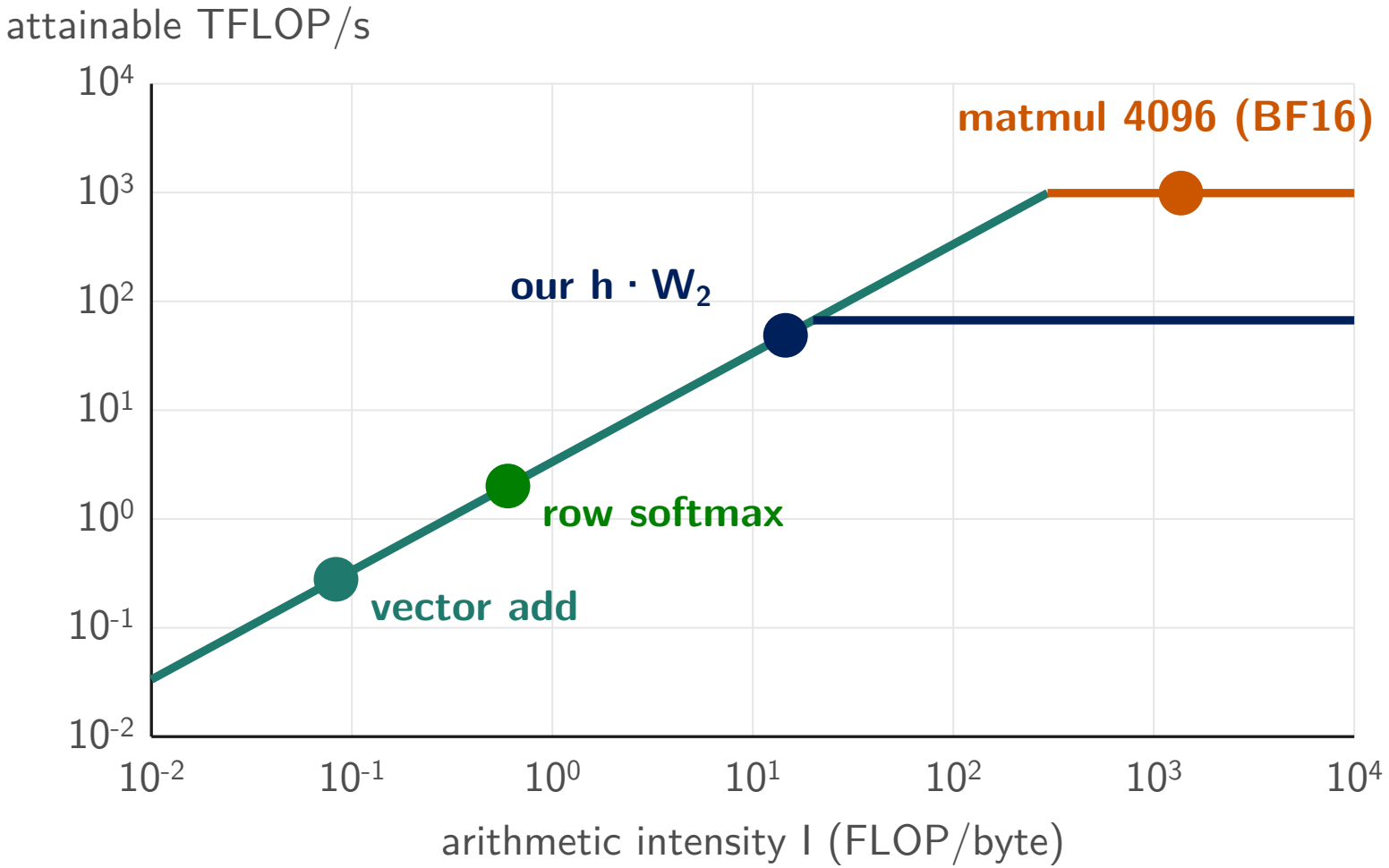
- Logits $h \cdot W_2$: $(64 \times 64) \cdot (64 \times 322)$, FP32
- $F \approx 2.64$ M FLOPs; $Q \approx 181$ KB; $I \approx 14.6$
- Roofline time ≈ 0.05 µs; whole step $\approx$ **0.17 µs**
- ≈ 25 kernel launches $\times \approx 4$ µs $\approx$ **100 µs**



In our pipeline

Overhead-bound: launches cost $\approx 600\times$ the roofline time. Fix: **fuse**, bigger batches, **compile**.

Placing kernels on the roofline



- **Tensor cores** help only the large matmul.
- **Fusion** helps the left side: it cuts Q.

Source: Williams, Waterman & Patterson, "Roofline," Communications of the ACM 52(4), 2009; H100 datasheet (SXM5).

Check yourself

Two minutes with a neighbour.

- A kernel has $I = 5$ FLOP/byte (FP32, H100). Which roof? Attainable TFLOP/s?
- A new GPU doubles π but keeps β. What happens to the ridge? Which kernels speed up?
- A kernel moves 1 GB and does 10^9 FLOPs (FP32). Predicted time?

Part 3

Using the roofline

Which operations are memory-bound, and what does the model miss?

Memory-bound and compute-bound operations

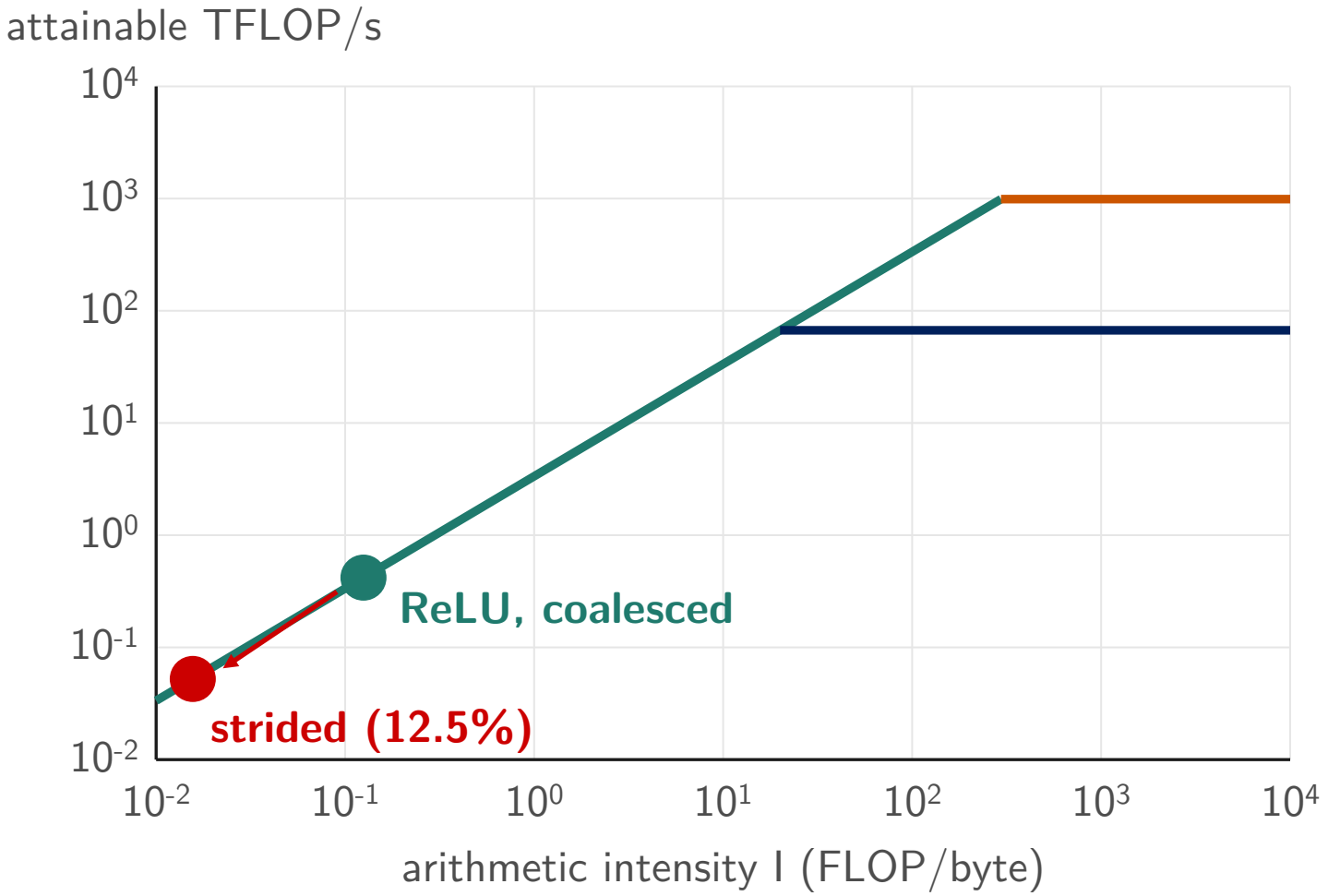
Operation (FP32 unless noted)	I (FLOP/B)	Bound
Elementwise: add, ReLU, tanh	≈ 0.1	memory
Reductions: sum, softmax	≈ 0.25–0.6	memory
Our logits matmul ($64 \times 64 \times 322$)	≈ 15	memory (just)
Large matmul, n $= 4096$ (BF16)	$\approx 1{,}365$	compute

Key idea

Most **operations** are memory-bound; most **FLOPs** sit in a few large matrix multiplies.

Source: Williams, Waterman & Patterson, "Roofline," Communications of the ACM 52(4), 2009; H100 datasheet (SXM5).

A bad access pattern moves you left



$$\text{real } Q = Q \,/\, \text{efficiency}$$

$$\text{real } I = I \times \text{efficiency}$$

$$1/8 \to 1/64\text{: } 8\times \text{ slower}$$

Watch out

Uncoalesced access **raises Q**: the kernel slides **left** and down the memory roof.

Source: Week 13, L1 sector model; Williams, Waterman & Patterson, "Roofline," Communications of the ACM 52(4), 2009; H100 datasheet (SXM5).

What the roofline leaves out

Effect	Symptom	Covered in
Launch / Python overhead	Small problems far below both roofs	Week 12, L2
Uncoalesced access	Q far above the minimum	Week 13, L1
Low occupancy	Latency not hidden; below the roof	Week 12, L2
L2 cache hits	Small inputs above the HBM roof	Week 13, L1

Key idea

When measurement and prediction disagree, the **gap is the diagnosis**.

Reading a measurement against the roof

$$\text{fraction of the roof} = \text{predicted time} \,/\, \text{measured time}$$

ReLU, n = 10^8 (illustrative)	**Predicted**	**Measured**	**Fraction**
A: coalesced	0.24 ms	0.27 ms	88%
B: something is wrong	0.24 ms	1.9 ms	13%

- **Near the roof** (A): done — the kernel cannot get much faster.
- **Far below** (B): find the cause in the table before changing code.

In our pipeline

Time it as in **Week 12, L2** (synchronize, warm up, median), then compare with the roof.

Check yourself

Two minutes with a neighbour.

- Softmax over a 4096 $\times$ 32,000 FP32 matrix: bound, and minimum time?
- Every access of your kernel has stride 2. How do I and the time change?
- A kernel on a 10 MB input runs 3$\times$ slower than the memory roof. Two suspects?

IN-CLASS BUILD

Predict a kernel bottleneck

Work in pairs. Predict first; measure in Lab 3.

Exercise: predict four kernels

Kernel (FP32)	I	Bound?	Time
1. ReLU $y = \max(x, 0)$, $n = 10^8$			
2. Row sums of a 4096×4096 matrix			
3. Matmul 2048×2048, ideal reuse			
4. Matrix $\times$ vector, 4096×4096			

USE

$\pi = 67$ TFLOP/s, $\beta = 3.35$ TB/s

Count F and Q first. Max = one FLOP.

STRETCH

Which kernels would tensor cores speed up? Predict kernel 3 in BF16.

Exercise: solutions

Kernel (FP32)	F; Q	I	Bound, time
1. ReLU, $n = 10^8$	10^8; 8.0×10^8 B	0.12	memory, ≈ 0.24 ms
2. Row sums, 4096^2	1.7×10^7; 6.7×10^7 B	0.25	memory, ≈ 20 µs
3. Matmul 2048	1.7×10^{10}; 5.0×10^7 B	341	compute, ≈ 0.26 ms
4. Matrix $\times$ vector	3.4×10^7; 6.7×10^7 B	0.5	memory, ≈ 20 µs

Stretch: only **kernel 3** gains from tensor cores. In BF16: $I \approx 683 > 295$, $\approx$ **17 µs**.

Key idea

Kernels 3 and 4 are both "linear algebra": only the one that **reuses** its inputs is compute-bound.

Checking your prediction in Lab 3

```
from triton.testing import do_bench
x = torch.randn(10**8, device="cuda")
ms = do_bench(lambda: torch.relu(x),
              return_mode="median")
pred = 8 * x.numel() / 3.35e12 * 1e3   # ms
print(f"{pred / ms:.0%} of the memory roof")
```

- Timing as in **Week 12, L2**: warm up, synchronize, median.
- Matmul: compare with **F** / π instead.
- Far below the roof? Use the **diagnosis** table.

In our pipeline

T1: next to your kernel, its **F, Q, I**, the **predicted** time, and the **measured** fraction of the roof.

Source: triton.testing.do_bench (Triton source); PyTorch documentation, CUDA semantics.

Key takeaways

- Count **F** (FLOPs) and the minimum **Q** (bytes to and from HBM); **I = F / Q**.
- Roofline: $\mathbf{T \geq \max(F/\pi,\ Q/\beta)}$; the **ridge** π/β is ≈ 20 (FP32) and ≈ 295 (BF16) FLOP/byte.
- Left of the ridge: **move fewer bytes**. Right of it: **faster arithmetic**.
- Elementwise ops and reductions are **memory-bound**; large matmuls are **compute-bound**.
- Our small MLP is **overhead-bound** — the motivation for fusion and compilation.
- The **gap** between prediction and measurement is the diagnosis.

Next time, and readings

When	What
Week 14, Lecture 1	Precision and the anatomy of a matrix multiply; then Lab 3 due
Week 14, Lecture 2	Triton, and one training step end to end
Fri, Apr 23 (Week 14)	T1 due

READ BEFORE WEEK 14

Williams, Waterman & Patterson, "Roofline," CACM 2009 (if not read yet).

CUDA C++ Programming Guide: "Shared Memory" and its matrix-multiplication example.